Table of Contents

1 Computer Architecture Assignment — Labs 9, 10 and 11

Student name: Raghav Bisht **Student ID:** 2023A7PS0484P **Course:** Computer Architecture CS-F342 **Simulator:** ChampSim

---

1.1 Abstract

This report evaluates branch predictors and last-level-cache configurations using ChampSim. Task 1 compares the existing bimodal, gshare, perceptron and hashed-perceptron predictors, a Modified Bullseye predictor and an adaptive-threshold Bullseye predictor. Task 2 evaluates LLC capacity, associativity and replacement policies, including implemented MRU, deterministic Random and adaptive Hard-to-Cache Identification Table (HCIT) policies.

1.2 Experimental methodology

- Single-core and four-core ChampSim configurations were evaluated.
- L1-I: 32 KB, 8-way, LRU.
- L1-D: 32 KB, 8-way, LRU.
- L2: 256 KB, 8-way, LRU.
- Default LLC: 4 MB, 16-way, LRU.
- Cache block size: 64 bytes.
- Four traces were used for every single-core configuration.
- The four-core mix was produced by the submitted seeded random trace generator.
- Warm-up: 1 million instructions; measured simulation: 10 million instructions.
- Predictor lookup latency is host-side average lookup time in nanoseconds and includes timer overhead.
- LLC writes are reported as LLC RFO accesses plus LLC writeback accesses; both are retained separately in detailed CSV files.

1.2.1 Block-size assumption

The assignment states that the 64-byte block size has an exception for Task 2(d), but no alternative block sizes are specified. The experiments therefore retain 64-byte blocks rather than inventing an unsupported configuration.

---

2 Task 1 — Branch Prediction

2.1 Existing and proposed predictors

The evaluated predictors are bimodal, gshare, perceptron, hashed perceptron, Modified Bullseye and adaptive-threshold Modified Bullseye.

2.2 Single-core predictor summary

Predictor	Average MPKI	Accuracy (%)	Average execution time (s)	Average IPC	Geomean IPC
bimodal	8.088125	96.4134	11.2775	0.875275	0.805069
gshare	6.278575	97.21105	10.9575	0.880607	0.811847
hashed_perceptron	5.02015	97.80495	11.26	0.899277	0.832926
perceptron	5.567325	97.556675	10.9625	0.892284	0.825273
modified_bullseye	5.3886	97.638075	12.1925	0.89578	0.828067
adaptive_bullseye	5.3846	97.64005	11.9575	0.895812	0.828116

2.3 Four-core predictor summary

Predictor	Average MPKI	Accuracy (%)	Average IPC	Geomean IPC
bimodal	2.800144	97.686775	0.816724	0.613247
gshare	2.297643	98.0982	0.823973	0.616378
hashed_perceptron	1.797575	98.4927	0.831506	0.622505
perceptron	2.053415	98.2961	0.826421	0.618927
modified_bullseye	1.884249	98.422	0.830004	0.621252
adaptive_bullseye	1.870349	98.43185	0.829696	0.620778

2.4 Modified Bullseye design

2.5 Assignment interpretation

The assignment specifies the following simplified control flow:

1. Predict using the existing hashed-perceptron predictor.
2. If the base prediction is wrong, increment a miss counter for that branch PC.
3. When the miss count exceeds a fixed threshold, place the PC in an H2P table.
4. Use a dedicated per-PC mini perceptron for H2P-resident branches.
5. Override the base prediction with the mini-perceptron prediction.

This implementation follows that requested pattern. It is inspired by the Bullseye paper but does not claim to reproduce the paper's full TAGE-SC-L, HIT trial, dual local/global perceptron, and admission system.

2.6 Structures

2.6.1 Base predictor

- 16-table hashed perceptron
- Geometric global-history lengths up to 232 branches
- Dynamic training threshold inherited from the supplied ChampSim predictor

2.6.2 Miss tracker

- 256 sets
- 4 ways
- LRU-style replacement
- Saturating per-PC base-misprediction count

2.6.3 Fixed H2P admission threshold

- 64 base-predictor mispredictions

2.6.4 H2P table

- 16 sets
- 4 ways
- 64 total H2P entries
- LRU-style replacement
- One dedicated 24-history-bit mini perceptron per resident PC

2.6.5 Prediction

- Non-H2P PC: hashed-perceptron output
- H2P-resident PC: mini-perceptron output overrides the base output

2.6.6 Training

- The base predictor continues training for every branch.
- The mini perceptron trains on a misprediction or a low-confidence correct prediction.
- Actual outcomes update both the base global history and mini-perceptron history.

2.7 Comparison target

The fixed-threshold predictor will be evaluated using the same:

- four single-core traces,
- fixed four-core trace mix,
- warm-up length,
- simulation length,
- cache hierarchy,
- measured lookup-latency instrumentation,

- parsing and graphing methodology

used for bimodal, gshare, hashed perceptron, and perceptron.

2.8 Adaptive-threshold Bullseye design

2.9 Goal

The fixed Modified Bullseye predictor admits a branch PC into the H2P table after a fixed number of base-predictor misses. The assignment additionally asks for a predictor that changes its threshold while the simulation is running.

This implementation retains the same hashed-perceptron base and dedicated H2P mini perceptrons, but changes the H2P admission threshold independently for each core.

2.10 Starting configuration

- Initial admission threshold: 64 base-predictor misses
- Minimum threshold: 16
- Maximum threshold: 128
- Adjustment step: 8
- Adaptation epoch: 8,192 observed branches

2.11 Signals observed per epoch

1. Recent base-predictor miss rate
2. Number of H2P overrides
3. Accuracy of H2P overrides
4. Current H2P occupancy

2.12 Threshold policy

2.12.1 Lower the threshold

The threshold is reduced by 8 when:

- recent base-predictor miss rate is above 10%, and
- the H2P layer has insufficient samples or at least 75% override accuracy.

This admits difficult branches sooner when the base predictor is struggling and the H2P mechanism appears useful.

2.12.2 Raise the threshold

The threshold is increased by 8 when either:

- H2P has at least 128 recent override samples but accuracy is below 55%;
- H2P occupancy is at least 87.5% while the recent base miss rate is below 10%;
- recent base-predictor miss rate is below 4%.

This conserves H2P capacity when the base predictor already performs well, the H2P overrides are weak, or table pressure is high.

2.13 Bounded adaptation

The threshold is clamped to the range 16–128. Epoch counters are reset after each adaptation decision, while predictor state and per-PC miss counters remain live.

2.14 Final instrumentation

At program exit, the predictor prints for every core:

- final admission threshold;
- number of threshold adjustments;
- base-predictor miss rate;
- H2P occupancy;
- H2P admissions;
- H2P overrides;
- H2P override accuracy.

These fields provide direct evidence that the threshold changed during the simulation and help explain the measured MPKI, accuracy, lookup latency, IPC, and simulation-time results.

2.15 Fair comparison

The adaptive predictor will use the same:

- four single-core traces;
- fixed randomly generated four-core trace mix;
- warm-up and simulation lengths;
- cache hierarchy;
- disabled prefetchers;
- LLC LRU policy;
- latency instrumentation;
- result parsing and plotting methodology

used for all earlier Task 1 predictors.

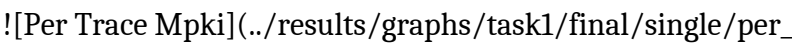
2.16 Task 1 graph results

2.16.1 Selected branch-prediction graphs

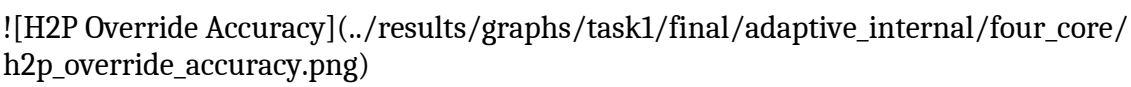
Average Mpki

Per Core Mpki

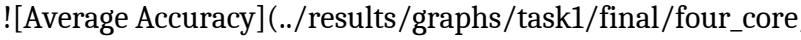
Per Trace Mpki



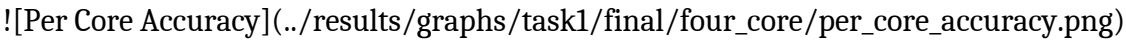
H2P Override Accuracy



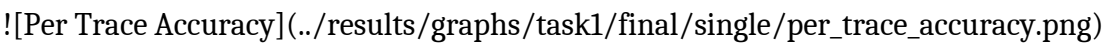
Average Accuracy



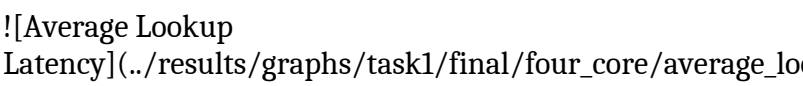
Per Core Accuracy



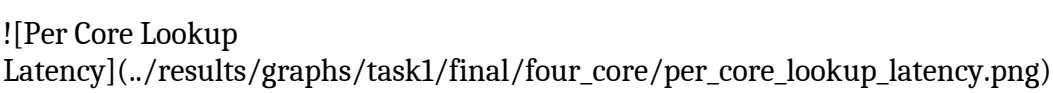
Per Trace Accuracy



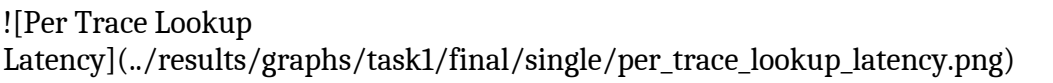
Average Lookup Latency



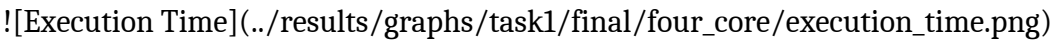
Per Core Lookup Latency



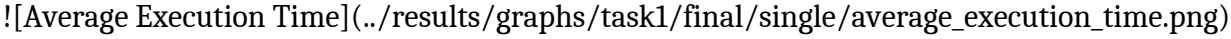
Per Trace Lookup Latency



Execution Time



Average Execution Time



2.17 Task 1 conclusions

Prediction quality and predictor complexity must be considered together. A lower MPKI and higher accuracy can improve IPC, but a larger predictor may increase lookup latency and simulation overhead. The Modified Bullseye design concentrates additional resources on difficult branches, while the adaptive version changes its threshold according to observed usefulness and occupancy.

---

3 Task 2 — Last-Level Cache

3.1 LLC size, associativity and replacement-policy analysis

3.2 Experimental basis

- L1-I: 32 KB, 8-way, LRU
- L1-D: 32 KB, 8-way, LRU
- L2: 256 KB, 8-way, LRU
- Default LLC: 4 MB, 16-way
- Block size: 64 bytes
- Branch predictor held constant: hashed perceptron
- Four single-core traces per configuration
- One seeded four-program mix for four-core experiments

LLC-write definition used in the tables: `LLC RFO accesses + LLC WRITEBACK accesses`. These fields are also retained separately in the detailed CSV files.

3.3 1. LLC Size Study: 1 MB, 2 MB, 4 MB and 8 MB

All configurations use a 16-way LLC, LRU replacement and the hashed-perceptron branch predictor.

3.3.1 Single-core summary

LLC size	Geomean IPC	L1 miss %	L2 miss %	LLC miss %	LLC hit %	LLC writes	Time (s)
1 MB	0.697613	10.6261	51.4193	25.1881	74.8119	1,430,619	15.350
2 MB	0.775963	10.6380	51.4191	20.2299	79.7701	1,430,620	14.912
4 MB	0.832926	10.6458	51.4189	18.4076	81.5924	1,430,621	14.320
8 MB	0.852731	10.6480	51.4189	18.1427	81.8573	1,430,622	17.688

3.3.2 Four-core summary

LLC size	Sum IPC	Geomean IPC	L1 miss %	L2 miss %	LLC miss %	LLC hit %	LLC writes	Time (s)
1 MB	2.115899	0.366720	7.8964	47.1916	92.7976	7.2024	25,646.00	787.520
2 MB	3.216118	0.591101	7.9274	47.1864	73.1836	26.8164	25,349.00	540.080
4 MB	3.326025	0.622505	7.9282	47.2070	45.5794	54.4206	25,851.00	473.710

LLC size	Sum IPC	Geomean IPC	L1 miss %	L2 miss %	LLC miss %	LLC hit %	LLC writes	Time (s)
8 MB	3.594913	0.685192	7.9248	47.2330	24.7394	75.2606	26,036.00	386.640

3.3.3 Main observations

- The best single-core geometric-mean IPC occurs at **8 MB**, while the lowest weighted single-core LLC miss percentage occurs at **8 MB**.
- Moving from 1 MB to 8 MB changes single-core geometric-mean IPC by **+22.236%** and LLC miss percentage by **-27.971%**.
- The best four-core aggregate IPC occurs at **8 MB**, while the lowest shared-LLC miss percentage occurs at **8 MB**.
- Moving from 1 MB to 8 MB changes four-core aggregate IPC by **+69.900%** and shared-LLC miss percentage by **-73.340%**.
- L1 and L2 miss percentages are included for completeness, but LLC capacity changes primarily affect the shared last-level cache. Any L1/L2 variation is indirect, caused by timing, replacement interactions and the set of requests reaching the LLC.

3.4 2. LLC Associativity Study: 16-way vs 8-way

The LLC capacity is held at 4 MB. Therefore, the 16-way configuration uses 4096 sets and the 8-way configuration uses 8192 sets, both with 64-byte blocks.

3.4.1 Single-core comparison

Associativity	Geomean IPC	LLC miss %	LLC hit %	LLC writes	Time (s)
16-way	0.832926	18.4076	81.5924	1,430,621	14.320
8-way	0.832847	18.4412	81.5588	1,430,619	11.893

3.4.2 Four-core comparison

Associativity	Sum IPC	LLC miss %	LLC hit %	LLC writes	Time (s)
16-way	3.326025	45.5794	54.4206	25,851.00	473.710
8-way	3.319878	47.4394	52.5606	26,565.00	400.420

3.4.3 Main observations

- Reducing associativity from 16-way to 8-way produces a **negligible** single-core geometric-mean IPC change of **-0.009%**.
- The weighted single-core LLC miss percentage increased by **+0.183%**.
- Four-core aggregate IPC changes by **-0.185%**, while shared-LLC miss percentage changes by **+4.081%**.
- At fixed capacity, lower associativity reduces the number of candidate locations available to each address and can increase conflict misses. However, doubling the

number of sets changes address mapping, so particular workloads may show little loss or even a small improvement. The measured workload results, rather than associativity alone, determine the final conclusion.

3.5 3. Replacement-Policy × LLC-Size Study

The comparison includes LRU, SRRIP, DRRIP, SHIP, the implemented MRU policy and the implemented deterministic Random policy.

3.5.1 IPC winners by LLC size

LLC size	Single-core winner	Geomean IPC	Four-core winner	Sum IPC
1 MB	LRU	0.697613	SRRIP	2.904598
2 MB	LRU	0.775963	SHIP	3.267608
4 MB	LRU	0.832926	SHIP	3.390684
8 MB	SRRIP	0.854861	SHIP	3.623260

3.5.2 Cross-size policy consistency

Policy	Avg single rank	Avg four-core rank	Mean single geomean IPC	Mean four-core sum IPC
LRU	1.25	2.75	0.789808	3.063239
SHIP	2.75	1.25	0.787901	3.284071
SRRIP	2.75	3.00	0.786920	3.230460
DRRIP	3.25	3.00	0.786454	3.175593
Random	5.25	5.00	0.763528	2.848260
MRU	5.75	6.00	0.732199	1.784757

3.5.3 Main observations

- **LRU** has the best average single-core IPC rank across the four LLC sizes.
- **SHIP** has the best average four-core aggregate-IPC rank across the four LLC sizes.
- At 1 MB, MRU changes IPC relative to LRU by **-10.503%** in single-core and **-25.176%** in four-core; Random changes it by **-4.713%** and **-5.707%**, respectively.
- At 2 MB, MRU changes IPC relative to LRU by **-12.015%** in single-core and **-50.140%** in four-core; Random changes it by **-5.291%** and **-14.344%**, respectively.
- At 4 MB, MRU changes IPC relative to LRU by **-6.809%** in single-core and **-50.773%** in four-core; Random changes it by **-2.674%** and **-5.861%**, respectively.
- At 8 MB, MRU changes IPC relative to LRU by **-0.847%** in single-core and **-35.604%** in four-core; Random changes it by **-1.045%** and **-2.306%**, respectively.
- LRU exploits temporal locality, SRRIP/DRRIP estimate re-reference distance, and SHIP uses PC-based reuse prediction. MRU intentionally discards the newest line, while Random maintains no locality state. Differences between policies are therefore expected to be workload- and capacity-dependent.

- A policy should not be declared universally best from IPC alone. LLC miss rate, hit rate, writes and execution time must be checked for the same configuration, especially when policies have similar IPC.

3.6 4. Report-ready overall conclusion

Increasing LLC capacity generally provides more opportunity to retain reusable blocks, but the measured IPC benefit depends on whether the selected workloads are capacity- or conflict-sensitive. The LLC miss and hit-rate trends should therefore be interpreted together with IPC rather than in isolation.

Reducing associativity from 16-way to 8-way at fixed 4 MB capacity changes both the number of ways and the number of sets. The measured result captures the combined effect of reduced placement flexibility and changed address mapping.

The replacement-policy study demonstrates that no policy is automatically optimal at every capacity and workload mix. Re-reference-prediction policies can outperform simple recency when they identify low-reuse lines, while LRU remains a strong baseline. MRU and Random provide useful contrast because they intentionally remove or ignore temporal-locality information.

3.7 5. Graph locations

- LLC size: `results/graphs/task2/llc\\\_size/`
- Associativity: `results/graphs/task2/llc\\\_associativity/`
- Replacement policy × size: `results/graphs/task2/replacement\\\_policy\\\_size/`

3.8 6. Methodology note

The assignment states that the 64-byte block size has an exception for Task 2(d), but it does not provide alternative block sizes. Therefore, the experiments retain the specified 64-byte block size and document this assumption rather than inventing an unsupported configuration.

3.9 MRU and Random implementations

3.10 Interface compatibility

Both policies implement the exact replacement interface used by this ChampSim version:

- `CACHE::llc\\\_initialize\\\_replacement()`
- `CACHE::llc\\\_find\\\_victim(...)`
- `CACHE::llc\\\_update\\\_replacement\\\_state(...)`
- `CACHE::llc\\\_replacement\\\_final\\\_stats()`

The build script copies the selected `.llc\\\_repl` file to `replacement/llc\\\_replacement.cc` before compilation.

3.11 MRU

3.11.1 State

MRU stores one way index per LLC set. The index identifies the block that was accessed most recently.

3.11.2 Victim selection

1. Select an invalid way when one exists.
2. Otherwise evict the stored most-recently-used way.

3.11.3 State update

Every LLC hit and fill marks the accessed way as most recently used, except for writeback hits. This exception matches the behavior of the supplied LRU policy.

3.11.4 Expected behavior

MRU deliberately evicts the newest block. It can perform poorly on workloads with strong temporal locality, but may help particular cyclic or scan-like access patterns where the newest block is not reused soon.

3.12 Random

3.12.1 State

Random uses one deterministic xorshift64* pseudo-random generator per core.

3.12.2 Victim selection

1. Select an invalid way when one exists.
2. Otherwise choose a way uniformly using `random\\\_value % LLC\\\_WAY`.

3.12.3 Reproducibility

Fixed seeds are used instead of `srand(time(NULL))`. Repeated simulations with the same inputs therefore use the same replacement sequence.

3.12.4 State update

No per-block recency or prediction state is maintained.

3.13 Final statistics

Each implementation prints its invalid-way and valid-victim selection counts. MRU additionally prints the number of recency-state updates.

3.14 Adaptive HCIT design

3.15 Objective

The policy is inspired by the observation that a small subset of branches can account for most branch-prediction misses. It applies the same idea to the LLC: only repeatedly difficult PCs or memory regions receive detailed tracking and preferential cache protection.

3.16 1. Candidate identification

Every LLC request is mapped to a signature:

- Demand, RFO and prefetch requests use a hash of the instruction PC.
- Writebacks, which do not carry a useful PC in this simulator, use a hashed 4-KB memory-region signature.

A 4096-entry, 4-bit candidate filter tracks recent miss pressure for all signatures. The filter is deliberately small and approximate. It prevents the larger HCIT from being allocated to ordinary one-off misses.

3.17 2. HCIT structure

The detailed HCIT contains 2048 entries arranged as 512 sets × 4 ways. An entry is allocated only after its candidate-filter pressure reaches the current adaptive admission threshold.

Each HCIT entry stores:

- signature,
- misses,
- insertions,
- hits,
- evictions,
- hardness score,
- last-touch timestamp,
- stale-epoch count.

This directly satisfies the requirement to track misses, insertions, hits and evictions per PC or memory region.

3.18 3. Hardness metric

At the end of every 16,384 LLC updates, counters are decayed and the score is recomputed as:

```
hardness = clamp(2×misses + 2×evictions + insertions − 2×hits, 0, 31)
```

Misses and evictions increase difficulty. Hits decrease difficulty because they show that the block is now being retained and reused successfully.

An HCIT entry is considered hard when:

```
score >= adaptive admission threshold
```

3.19 4. Adaptive admission and removal

The admission threshold begins at 6 and is restricted to the range 3–12.

It increases when:

- HCIT occupancy is high and protected-block utility is low, or
- protected evictions substantially exceed protected hits.

It decreases when:

- HCIT occupancy is low and protected-block utility is high, or
- protected hits strongly exceed protected evictions.

HCIT sets replace their lowest-score entry, with oldest touch as a tie-breaker. A candidate is rejected when it is clearly weaker than the selected HCIT victim.

Old entries leave the HCIT after repeated low-score, low-activity epochs.

3.20 5. Adaptive protection mechanism

The cache itself uses a 2-bit RRIP state:

- Normal insertion: RRPV = 2.
- Hard-candidate insertion: RRPV = 0.
- Cache hit: RRPV = 0.

Victim selection first searches for an unprotected block with maximum RRPV. Unprotected blocks are aged before protected blocks. A protected block is selected only when all ways in the set are protected.

This combines:

- preferential insertion,
- delayed eviction,
- adaptive admission,
- adaptive removal.

3.21 6. Storage overhead

Approximate policy storage at the default 4-MB, 16-way LLC:

- RRIP state: 2 bits per LLC block.
- Per-block signature metadata: 64 bits plus one valid bit per LLC block.
- Candidate filter: 4096 × 4 bits.
- HCIT: 2048 detailed entries.

The large per-block signature is used here for implementation clarity and experimental correctness. A hardware implementation could store compressed signatures or tags to reduce overhead.

3.22 7. Printed statistics

The policy prints:

- final adaptive threshold,
- threshold increases/decreases,
- HCIT occupancy,
- admissions and rejected admissions,
- HCIT replacements and stale removals,
- protected and normal fills,
- protected hits and evictions,
- protection utility,
- protected/unprotected victim selections.

3.23 Adaptive HCIT experimental analysis

HCIT is evaluated as the seventh replacement policy at 1 MB, 2 MB, 4 MB and 8 MB.

3.24 Performance comparison

Size	HCIT single IPC	vs LRU	Best baseline	vs best	HCIT four sum IPC	vs LRU	Best baseline	vs best
1 MB	0.69355	-0.582 %	LRU	-0.582 %	2.19811	+3.885 %	SRRIP	-24.323 %
2 MB	0.75901	-2.185 %	LRU	-2.185 %	2.62133	-18.494 %	SHIP	-19.778 %
4 MB	0.81919	-1.649 %	LRU	-1.649 %	2.83089	-14.887 %	SHIP	-16.510 %
8 MB	0.84820	-0.531 %	SRRIP	-0.779 %	3.20644	-10.806 %	SHIP	-11.504 %

3.25 HCIT internal behavior

Size	Single threshold	Single occupancy	Single utility	Four threshold	Four occupancy	Four utility
1 MB	6.25	1.39%	48.40%	3	27.20%	49.64%

Size	Single threshold	Single occupancy	Single utility	Four threshold	Four occupancy	Four utility
2 MB	5.25	1.45%	60.07%	3	18.75%	59.63%
4 MB	5.25	1.45%	68.58%	3	16.80%	57.58%
8 MB	5.25	1.38%	77.70%	3	17.38%	57.40%

3.26 Conclusions

- HCIT allocates detailed tracking only after the candidate filter identifies repeated miss pressure.
- The final threshold and occupancy values show whether the adaptive controller became more or less selective at each capacity.
- Protected utility is the fraction of protected-block outcomes that were hits rather than evictions.
- HCIT should be judged against both LRU and the strongest conventional policy at each size; a single workload mix cannot establish universal superiority.
- Even when IPC is similar, HCIT remains a valid bonus design because it implements selective hard-candidate tracking, adaptive admission/removal, preferential insertion and delayed eviction.

3.27 Graph locations

- `results/graphs/task2/hcit\\\_comparison/single/`
- `results/graphs/task2/hcit\\\_comparison/four\\\_core/`

3.28 Task 2 graph results

3.28.1 LLC-size graphs

Geomean Ipc

Llc Miss Percent

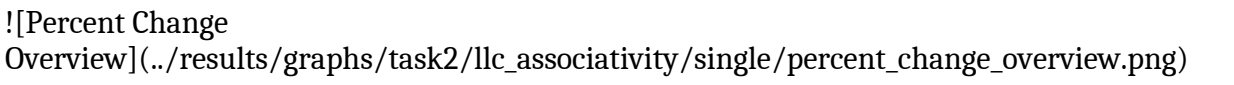
Sum Ipc

Llc Miss Percent

3.28.2 Associativity graphs

Geomean Ipc

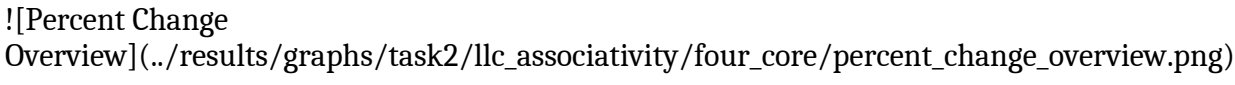
Percent Change Overview



Sum Ipc

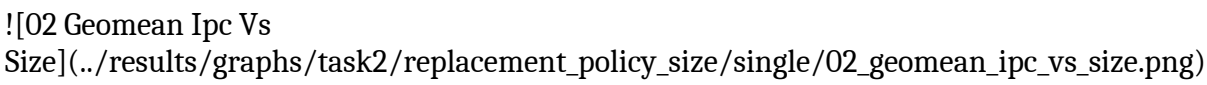


Percent Change Overview

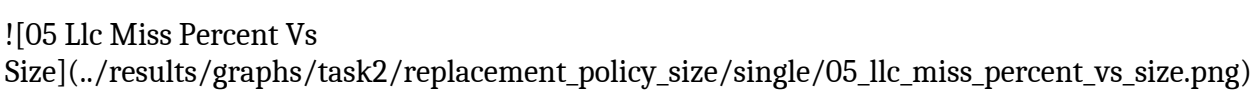


3.28.3 Replacement-policy and size graphs

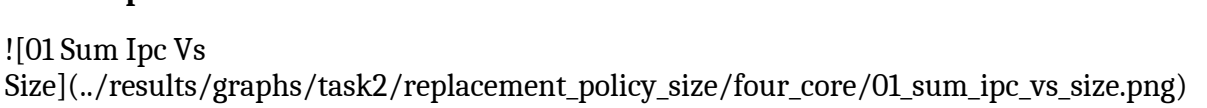
02 Geomean Ipc Vs Size



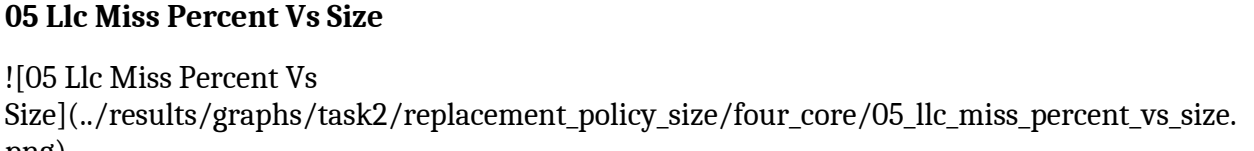
05 Llc Miss Percent Vs Size



01 Sum Ipc Vs Size

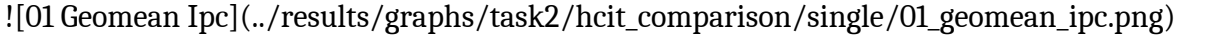


05 Llc Miss Percent Vs Size

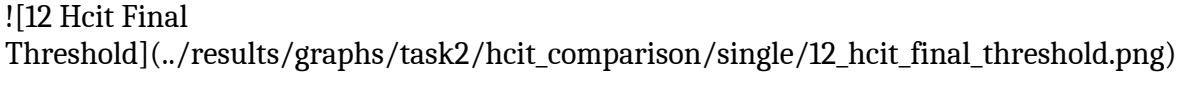


3.28.4 HCIT graphs

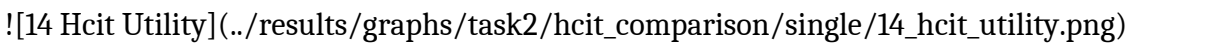
01 Geomean Ipc



12 Hcit Final Threshold



14 Hcit Utility



01 Sum Ipc

![01 Sum Ipc](../results/graphs/task2/hcit_comparison/four_core/01_sum_ipc.png)

12 Hcit Final Threshold

![12 Hcit Final Threshold](../results/graphs/task2/hcit_comparison/four_core/12_hcit_final_threshold.png)

14 Hcit Utility

![14 Hcit Utility](../results/graphs/task2/hcit_comparison/four_core/14_hcit_utility.png)

3.29 Overall conclusions

LLC capacity, associativity and replacement policy interact with workload locality. Larger capacity generally reduces capacity misses, while lower associativity can alter conflict behaviour and address mapping. LRU remains a strong baseline; RRIP-family and PC-correlated policies can improve results when they identify low-reuse lines. MRU and Random demonstrate the cost or occasional benefit of discarding locality information.

HCIT selectively allocates detailed tracking to candidates showing persistent miss pressure. It combines adaptive admission and removal with preferential RRIP insertion and delayed eviction. Its internal threshold, occupancy and protection-utility statistics demonstrate that the policy changes selectivity according to observed behaviour.

---

4 Reproducibility and submitted material

- Predictor source files are stored in `branch/`.
- Replacement-policy source files are stored in `replacement/`.
- Experiment runners, parsers, plotting and analysis scripts are stored in `scripts/`.
- Raw outputs and timing files are stored in `results/raw/`.
- Parsed tables are stored in `results/csv/`.
- Graphs are stored in `results/graphs/`.
- Configuration records are stored in `configs/`.
- Random-trace generator evidence is stored in `random\\\_trace\\\_output/`.
- Supporting design documents and the submission checklist are stored in `reports/`.

5 Appendix — Important result files

- `results/csv/task1\\\_single\\\_summary\\\_all\\\_predictors.csv`
- `results/csv/task1\\\_four\\\_core\\\_summary\\\_all\\\_predictors.csv`
- `results/csv/task2\\\_llc\\\_size\\\_single\\\_summary.csv`
- `results/csv/task2\\\_llc\\\_size\\\_four\\\_core\\\_summary.csv`
- `results/csv/task2\\\_llc\\\_associativity\\\_single\\\_summary.csv`

- results/csv/task2_llc_associativity_four_core_summary.csv
- results/csv/task2_replacement_policy_size_single_summary_with_hcit.csv
- results/csv/task2_replacement_policy_size_four_core_summary_with_hcit.csv
- results/csv/task2_hcit_internal_single.csv
- results/csv/task2_hcit_internal_four_core.csv